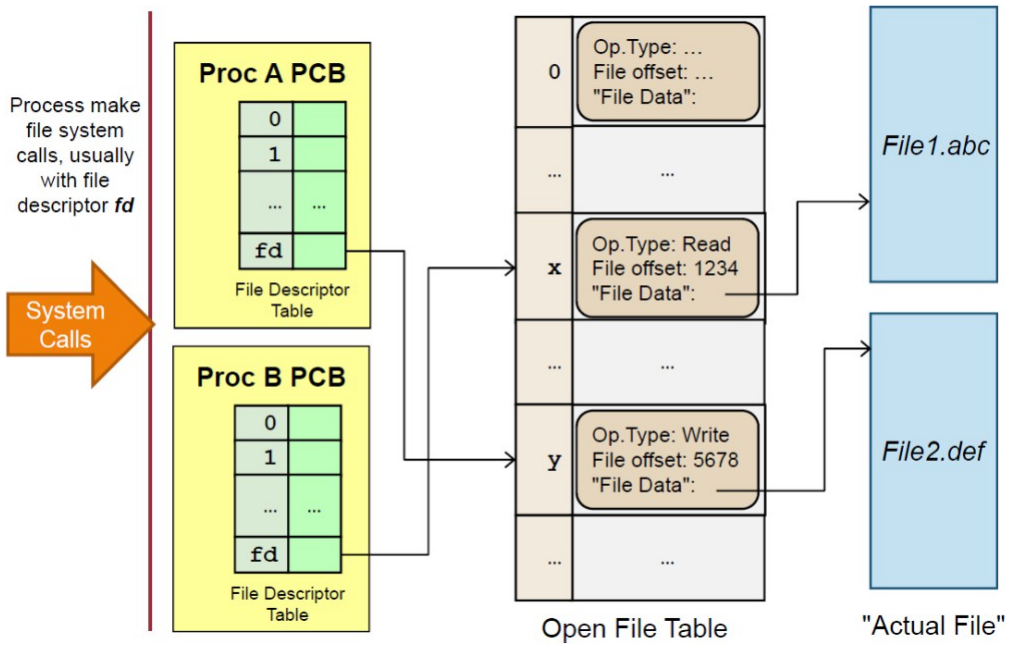
IT5002 Computer Systems and Applications
Tutorial 10
SUGGESTED SOLUTIONS

1. [Open File Table] Below is an illustration of how an OS organizes file data at runtime:



Discuss how this organization helps OS to handle the following scenarios. Your answer should refer to the relevant structure(s) if possible.

a. Process A tries to open a file that is currently being written by Process B.

OS uses the Open File Table to check for existing opened file. Since the file is already opened for reading, it can reject the file open system call from process A.

b. Process A tries to use a bogus file descriptor in a file-related system call.

Since Process A passed the file descriptor (fd for short) to OS as parameter, OS can check whether that particular entry is valid (or even exists) in the PCB of A. If the fd is out of range, non-existent etc, OS can reject the file-related system calls made by Process A.

c. Process A can never "accidentally" access files opened by Process B.

Since the fd index into process specific PCB, there is no way Process A can access Process B's file descriptor table.

d. Process A and Process B reads from the same file. However, their reading should not affect each other.

Process A and Process B can have their own **fd**s, which refers to two distinct locations in the open file table. Each entry of the open file table keep track of the current location separately. This enables Process A and Process B to read from the same file independently.

e. Redirect Process A's standard input / output, e.g. "**`a.out < test.in > test.out`**". (Hint: *nix processes has 3 default file descriptors: 0 = stdin, 1 = stdout, 2 = stderr)

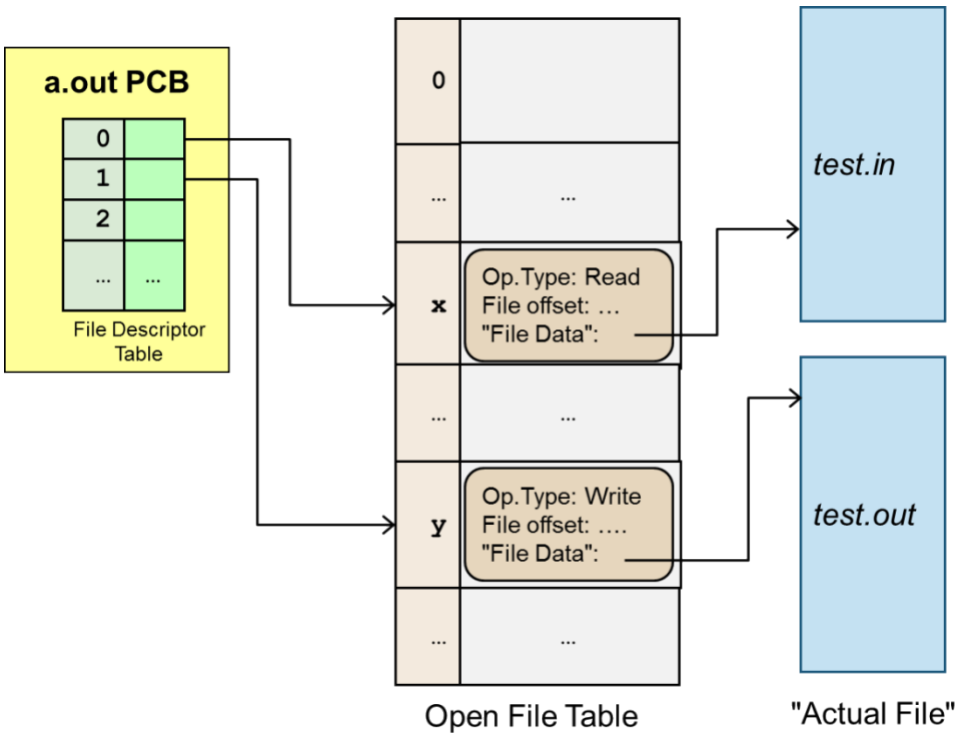
Key points: There are 3 standard file descriptors for every program in Unix (0 = stdin, 1 = stdout, 2 = stderr). These are "opened" automatically and linked to the corresponding files. Note that screen (terminal), keyboard are represented as *special files* in unix.

So, for all file redirections, it is a simple question of:

a. Opening and possibly creating the file.
b. Replace the corresponding file descriptor to point to the entry from (1) in the open file table.

The following illustrate the last case:

a.out < test.in > test.out



2. [Putting it together] This question is based on a "simple" file system built from the various components discussed in lecture 12.

Partition Information (Free Space Information)

- Bitmap is used, with a total of 32 file data blocks (1 = Free, 0 = Occupied):

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
0	0	0	0	0	1	0	0	1	0	0	0	0	1	0	1
16	**17**	**18**	**19**	**20**	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**	**31**
1	0	1	0	1	0	1	0	1	1	1	0	0	1	0	0

Directory Structure + File Information:

- Directory structures are stored in 4 "directory" blocks. Directory entries (both files and subdirectories) of a directory are stored in a single directory block.
- Directory entry:
  - **For File:** Indicates the first and last data block number.
  - **For Subdirectory:** Indicates the directory structure block number that contains the subdirectory's directory entries.
  - The "**/**" root directory has the directory block number 0.

0	**1**	**2**	**3**
y |Dir | 3 f |File| 12| 2 x |Dir | 1	g |File| 0| 31 z |Dir | 2	k |File| 6| 6	i |File| 1| 3 h |File|27|28

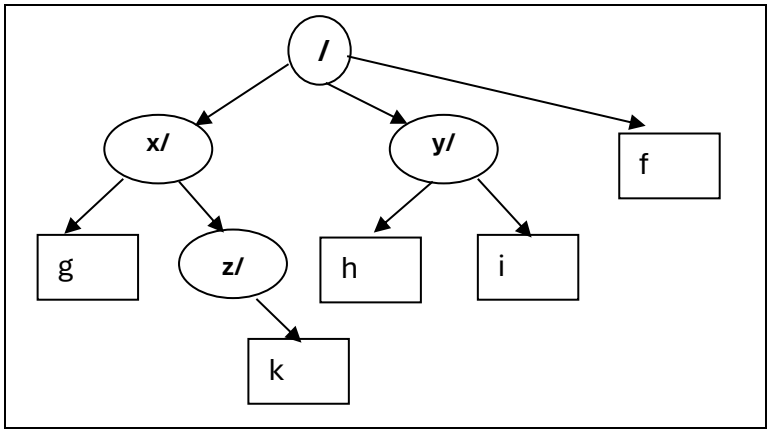
File Data:

- Linked list allocation is used. The first value in the data block is the "next" block pointer, with "-1" to indicate the end of data block.
- Each data block is 1 KB. For simplicity, we show only a couple of letters/numbers in each block.

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**
11 AL	9 TH	-1 S!	-1 ND	23 GS	-1 SO	-1 :)	10 TE
8	**9**	**10**	**11**	**12**	**13**	**14**	**15**
31 RE	3 EE	28 M:	31 OH	19 SE	13 AH	4 IN	17 NO
16	**17**	**18**	**19**	**20**	**21**	**22**	**23**
30 YE	2 OU	1 ON	17 RI	26 EV	14 AT	21 DA	7 YS
24	**25**	**26**	**27**	**28**	**29**	**30**	**31**
-1 HO	18 ME	0 AL	30 OP	-1 -(	5 LO	21 ER	-1 A!

a. (Basic Info) Give:
   - The current free capacity of the disk.
   - The current user view of the directory structure.

- **~12KB free space (12 '1' in Bitmap, each data block is 1KB). Due to the linked list file allocation overhead, the actual capacity is a little bit smaller.**



b. (File Paths) Walkthrough the file path checking for:
   - "**/y/i**"
   - "**/x/z/i**"

Only the directory block number is indicated:
- **/y/i**: 0, 3 (successful)
- **/x/z/i**: 0, 1, 2 (failed)

c. (File access) Access the entire content for the following files:
   - "**/x/z/k**"
   - "**/y/h**"

- **/x/z/k**: block 6, content = ":)"
- **/y/h**: blocks 27, 30, 21, 14, 4, 23, 7, 10, 28, content = "OPERATINGSYSTEM:-("

d. (Create file) Add a new file "**/y/n**" with 5 blocks of content. You can assume we always use the free block with the smallest block number. Indicate **all changes required to add the file.**

Bitmap updated: Bit 5, 8, 13, 15, 16 changed to 0
Directory block 3 (for /y) updated: "n |File| 5|16" added
Data Blocks 5, 8, 13, 15, 16 (next block pointer changed, with -1 in block 16).

3. Most OSes perform some higher-level I/O scheduling on top of just trying to minimize hard disk seeking time. For example, one common hard disk I/O scheduling algorithm is described below:

   i. User processes submit file operation requests in the form of
      ***operation*(starting hard disk sector, number of bytes)**
   ii. The OS sorts the requests by hard disk sector.
   iii. The OS merges requests that are nearby into a larger request, e.g. several requests asking for tens of bytes from nearby sectors merged into a request that reads several nearby sectors.
   iv. The OS then issues the processed requests when the hard disk is ready.

   a. How should we decide whether to merge two user requests? Suggest two simple criteria.

      Criterion 1: Requests are in the same or nearby sector (can mention cluster size).
      Criterion 2: Requests are of the same type, read / write.

   b. Give one advantage of the algorithm as described.

      Advantage: Seeking latency is reduced.

   c. Give one disadvantage of the algorithm and suggest one way to mitigate the issue.

      Disadvantage: Potential starvation for user process if the request is not near to existing requests.
      Mitigate: Take the request time into account and set certain deadline. Once the deadline is near, issue request regardless of whether it can be merged.

   d. Strangely enough, the OS tends to **intentionally delay** serving user disk I/O requests. Give one reason why this is actually beneficial using the algorithm in this question for illustration.

      Reason to delay: Disk I/O request has very high latency. Delaying the user request for several hundred machine instructions (note that instruction execution time is in the ns range) will not increase the waiting time significantly. However, with more user requests pending, OS can optimize the I/O better. If we do not have enough I/O requests to choose from, merging will not be very effective.

e. In modern hard disks, algorithms to minimise disk head seek time (e.g. SCAN variants, FCFS etc.) are **built into the** hardware controller. i.e. when multiple requests are received by the hard disk hardware controller, the requests will be reordered to minimise seek time. Briefly explain how the high-level I/O scheduling algorithm described in this question may conflict with the hard disk's built-in scheduling algorithm.

Potential conflict: It may turn out that the hard disk controller schedule the requests differently. In the worst case, the scheduling decision by OS may be undone by the controller time used for sorting / merging are wasted.